

DIGITAL FORENSICS LABORATORY · LAB 03

Document Analysis and Steganography

Working paper — Office file anatomy, macros, metadata, and hidden messages

Name

Student ID (NIM)

Class / Group

Date

Instructor / TA

Session grade

1. Lab Overview

This working paper accompanies Lab 03 of the digital forensics series. Documents are the most ordinary files in any organization - which makes them an ideal carrier for metadata leaks, malicious macros, and steganography. In this lab you take Office files apart as ZIP archives, read their XML with your own eyes, triage VBA macros without ever executing them, mine metadata with exiftool, and chase hidden messages through pixels and spectrograms. Complete the pre-lab before the session, check off each step as you perform it, answer every question with numbered evidence, and close with a reflection.

1.1 Learning objectives

- Explain the difference between OLE and OOXML, and why every .docx/.pptx is secretly a ZIP archive.
- Dissect an OOXML package: [Content_Types].xml, docProps, relationships, and the document part.
- Detect and read VBA macros with oleid and olevba - without running them.
- Recognize obfuscation families in macro code and decode them offline in Python.
- Extract metadata with exiftool: author, dates, software, and GPS coordinates (with DMS-to-decimal conversion).
- Detect PNG LSB steganography with zsteg, extract JPEG payloads with steghide, and read audio spectrograms in Audacity.

1.2 Environment and files

Environment: Kali Linux (or any Linux VM) with oletools, exiftool, zsteg, steghide, and Audacity installed; Python 3 for decode scripts. Windows users can do the OOXML drills anywhere a ZIP tool

exists, but the stego tools are Linux-side. Copy every evidence file to your working directory first - you never analyze originals in place, and you never open a macro-enabled document in Office.

Table 1.1 — Lab files

File	Purpose
files/YearlyBonus.docm	Challenge A — phishing document with an obfuscated macro secret (upstream)
files/Presentation.pptx	Challenge A — the recovered spy deck hiding the mole's photo (upstream)
files/super_secret_audio.wav	Challenge A — audio with a spectrogram flag (upstream)
files/starry_night.png	Guided drill — PNG with a known LSB message (upstream)
files/cover_letter.docx	Challenge B (new) — a job application whose XML tells a longer story
files/harbor_photo.png	Challenge C (new) — a stock photo whose metadata says 'nothing here'

1.3 What to submit

- Section 1.4 pre-lab, answered before the session (worth 10 points).
- This working paper as PDF, named NIM_Name_Lab03.pdf.
- Your evidence folder NIM_Lab03/evidence/ containing every numbered exhibit (B-01, B-02, and so on).
- Answers that cite exact part names, fields, commands, or tool output - not vague descriptions.

1.4 Pre-Lab Check

Answer these before the session, using the lab README and Section 2 of this paper as references. One or two sentences per question are enough - precision beats length.

P1. What is the difference between OLE and OOXML, and which file extensions belong to each family?

Answer:

Two sentences maximum.

P2. Why can a .docx be opened with unzip, and what forensic advantage does that give an investigator?

Answer:

Two sentences maximum.

P3. What is a VBA macro, which auto-run hook fires when a document is simply opened, and which part of a .docm stores the macro code?

Answer:

Two sentences maximum.

P4. Define steganography in one sentence and state how it differs from cryptography.

Answer:

Two sentences maximum.

P5. Name three kinds of information exiftool can reveal about a photograph.

Answer:

Two sentences maximum.

2. Theory Primer

2.1 OLE vs OOXML

Microsoft Office has used two container families. OLE (Object Linking and Embedding, 1997-2003) is a binary 'file within a file' structure (.doc, .ppt, .xls, .rtf). OOXML (Office Open XML, 2007 onward) stores the document as a ZIP archive of XML parts (.docx, .pptx, .xlsx; the m-suffix variants add VBA macros). For an investigator the ZIP design is a gift: any archive tool exposes the full document - text, properties, relationships, media - without opening Office and without executing anything.

Table 2.1 — The two Office container families

	OLE (1997-2003)	OOXML (2007-now)
Extensions	.doc .ppt .xls .rtf	.docx .pptx .xlsx (+ .docm etc. with macros)
Structure	binary compound file	ZIP archive of XML parts
Read with	oletools, OLE parsers	unzip + grep + xmllint
Macros live in	VBA storage streams	word/vbaProject.bin inside the ZIP

2.2 Anatomy of an OOXML package

Unzipping a Word document yields a small, predictable tree. Learn it once and every Office file becomes readable at a glance.

Table 2.2 — Parts that matter to an investigator

Part	Holds	Investigator's question
[Content_Types].xml	registry of part types and extensions	is anything unexpected declared?
docProps/core.xml	title, author, created/modified, revision	who wrote it and when?
docProps/app.xml	application, template, company, edit time	what software produced it?
word/document.xml	the visible text AND hidden text	is there more than we can see?
word/_rels/document.xml.rels	relationships: styles, images, EXTERNAL targets	does it phone home?
word/vbaProject.bin	compiled VBA macros (m-variants only)	what code runs on open?
ppt/slides/ + ppt/media/	slide XML and embedded media (pptx)	what was smuggled in?

Hidden text is the classic OOXML trick: a run marked `<w:vanish/>` is stored in document.xml but never displayed. Screen says one thing, XML says another. White-on-white text and tiny fonts achieve the same effect. The relationships files repay equal attention: a relationship with `TargetMode="External"` points outside the package - for example a remote template fetched from an attacker's server the moment the document opens.

2.3 Macros and oletools

A macro is a Visual Basic for Applications program stored inside the document. Legitimate offices use them for automation; attackers use them because one click on 'Enable Content' runs their code. Macros ride the auto-run hooks - `AutoOpen()/Document_Open()` fires on open, `AutoClose()` on close - and are stored compiled in `vbaProject.bin`, so a text editor is not enough. The oletools suite is the standard answer: `oleid` triages a file (format, encryption, macro presence, external relationships) and `olevba` extracts the full VBA source with a suspicious-keyword analysis (`AutoExec`, `Shell`, `CreateObject`, encoded strings). Never execute a macro to learn what it does - read it.

2.4 Steganography: images and audio

Steganography hides the existence of a message inside an innocent carrier. In PNG images the dominant technique flips the least significant bit (LSB) of selected pixel channels: a 200x200 photo offers 120,000 carry bits with changes no eye can detect - and because PNG is lossless, the bits survive re-saving. `zsteg` brute-forces bit planes (b1..b8), channel combinations (rgb, rgba, r, g, b), bit orders (lsb/msb) and scan orders (xy, YX); its output grammar 'b1,rgb,lsb,xy' names exactly what it found. JPEG is lossy, so raw LSBs die at the next save; `steghide` instead swaps pairs of DCT coefficients, optionally protected by a passphrase (try empty first). Audio carries messages either in LSBs or - more theatrically - as shapes drawn into the spectrogram: frequencies plotted over time that spell text when visualized in Audacity.

2.5 Metadata and exiftool

exiftool reads (and writes) metadata for images, audio, video, and Office files alike: Author/Artist, Software, creation and modification times, camera model, and GPS coordinates. GPS arrives in degrees-minutes-seconds; convert to decimal degrees with $\text{decimal} = \text{degrees} + \text{minutes}/60 + \text{seconds}/3600$, then paste into any map. Treat metadata as a lead, not a verdict - fields are editable - and corroborate before you accuse.

3. Guided Exercises

Work through the five drills in order. Every output is an exhibit: capture it, number it (B-01, B-02, and so on), and log it in Appendix B. Drills stop where the challenges begin: triage yes, decoding no.

3.A OOXML anatomy on your own file

- 1. Create a new Word document with the text 'Hello from Lab 03', save it as Me.docx, and copy it to your working directory.
- 2. Compute and record its MD5 and SHA1 (md5sum / sha1sum) before doing anything else.
- 3. `unzip -l Me.docx` - list the parts. Then `unzip Me.docx -d me_x/` and open `word/document.xml` in an editor; find your sentence.
- 4. Read `docProps/core.xml`: what creator, created, and modified values did your own machine record?
- 5. Add the same sentence again but formatted as hidden text (Home > Font > Hidden), re-save, and re-extract: does the hidden sentence appear in `document.xml`? Record the run's XML as an exhibit.

3.B Macro triage with oletools

- 1. `pip install oletools`, then run `oleid files/YearlyBonus.docm` (work on a copy). Record every indicator and its risk level.
- 2. Run `olevba` on the same copy. Which module and which auto-run hook does the macro use?
- 3. Record the suspicious keywords `olevba` flags. Do NOT decode the payload yet - that is Challenge A.
- 4. In one sentence: why is reading the macro with `olevba` safer than opening the file with macros enabled?

3.C zsteg on the known sample

- 1. Install `zsteg` (`gem install zsteg`) and run it against `files/starry_night.png`.
- 2. Record the `b1,rgb,lsb,xy` line verbatim - the fishermen quote.
- 3. Explain the other lines (`b2`, `b4`): real message, structured data, or noise?

- **4.** Re-save the PNG (convert `starry_night.png` to `out.png`) and re-run `zsteg`: does the message survive? Why?

3.D exiftool on your own photo

- **1.** Pick any photo you took with a phone (or download a camera-original sample). Run `exiftool` on it and record Artist/Author, Software, and dates.
- **2.** Run `exiftool -gps:all -a` on it. If GPS exists, convert to decimal degrees with the Section 2.5 formula and name the city on a map.
- **3.** Compare `exiftool` output for `Me.docx` from drill 3.A: which fields does Office fill in?

3.E Spectrogram warm-up (Audacity)

- **1.** Open Audacity; Generate > Tone, 1000 Hz, 10 seconds.
- **2.** Switch the track to Spectrogram view (track drop-down menu). What does a steady 1000 Hz tone look like?
- **3.** Generate a Chirp from 500 to 4000 Hz; view it. You now know how frequency maps to height.
- **4.** Optional: `sox tone.wav -n spectrogram -o spec.png` renders the same picture headless - try it.

4. Challenges

Challenge A is the upstream case - solved together in the Deep Dive session. Challenges B and C are yours: work on copies, hash before and after, cite part names and commands. Flags use `ctf{xxxx}` (Challenge A keeps the upstream flag format). Hints are staged - stop after each one and retry.

4.1 Challenge A — the upstream trio (solved in the Deep Dive)

The three evidence sets from Table 1.1: the phishing `.docm`, the spy's `.pptx`, and the `.wav`. The Deep Dive deck walks through the full solution - use your own exhibits, not the deck's screenshots. Your deliverables: the macro secret (flag format `flag{...}`), the source's name and city from the photo's metadata, the extracted payload method, and the spectrogram screenshot with its flag.

4.2 Challenge B — the cover letter (files/cover_letter.docx)

- **1.** A job application arrived at a small company. HR swears it 'looks completely clean'. Copy the file to your working directory and hash it.
- **2.** Do NOT trust the screen. Unzip the package and list every part. Anything that should not be in a plain cover letter?
- **3.** Read `word/document.xml` fully (`xmllint --format`). What is stored there that Word never displays? Quote the mechanism.

- **4.** Recover the hidden operational content: the staging URL, the drop instructions, and the auth token (the flag).
- **5.** Inspect `word/_rels/document.xml.rels`. One relationship has `TargetMode="External"` - write down its type and target, and explain the risk in one sentence.
- **6.** Read `docProps/core.xml` and `app.xml`. Who is the claimed author, and does anything contradict the visible letter?

Hints: (1) The visible letter is five short paragraphs - the XML has more paragraphs than that. (2) `grep -o 'vanish' word/document.xml | wc -l` tells you how many runs are hidden from view. (3) An honest letter has no reason to fetch a template from the internet.

Flag B: `ctf{_____}`

4.3 Challenge C — the harbor photo (`files/harbor_photo.png`)

- **1.** A 'stock archive' photo was attached to a shipping schedule email. Copy it, hash it, and run `exiftool` on it - record Author and Comment.
- **2.** The metadata says 'no copyright, nothing here'. Run `zsteg` against it and record every hit with its plane grammar.
- **3.** Identify the plane that carries a readable English message. What does the grammar string mean, field by field?
- **4.** Recover the full smuggling note: the drop point, the signal, and the auth token (the flag).
- **5.** The `tEXt` metadata was a decoy. In two sentences: why do attackers plant fake metadata, and what made you look past it here?

Hints: (1) `zsteg`'s first hit is usually the real one - read the `b1,rgb,lsb,xy` line. (2) 'file:' hits are structured data; 'text:' hits are messages. (3) A stock photo with a smuggling note inside is not a stock photo.

Flag C: `ctf{_____}`

5. Analysis Questions

Questions 1-3 come from the original lab; questions 4-12 extend the case. Cite exact parts, fields, commands, or tool output in every answer.

1. The phishing document (files/YearlyBonus.docm):

- a.** Which auto-run hook fires when the document is opened?
- b.** How is the secret obfuscated inside the macro, and how did you decode it offline?
- c.** State the secret in its original flag{...} format.

Answer:

Write your complete answer, including the exact file, part, or command used.

Supporting evidence (screenshot / output): exhibit numbers B-_____ to B-_____

2. The mole's presentation (files/Presentation.pptx):

- a.** Which embedded image is the smuggled one, and which file told you?
- b.** What is the name of the source inside the government, and which field gave it away?
- c.** Convert the photo's GPS position to decimal degrees and name the city. Show the conversion.
- d.** Which command extracts the top secret payload from the image, and under what conditions does it run?

Answer:

Write your complete answer, including the exact file, part, or command used.

Supporting evidence (screenshot / output): exhibit numbers B-_____ to B-_____

3. The audio file (files/super_secret_audio.wav):

- a.** List the exact steps to view the spectrogram in Audacity.
- b.** What does the spectrogram spell, in flag{...} format?

Answer:

Write your complete answer, including the exact file, part, or command used.

Supporting evidence (screenshot / output): exhibit numbers B-_____ to B-_____

4. Why is unzipping a .docx forensically safer than opening it in Word? Name one risk that still remains even when you only unzip.

Answer:

Write your complete answer, including the exact file, part, or command used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

5. Explain the difference between hidden text and deleted text in a Word document: where does each survive, and which one can still be recovered from the file itself?

Answer:

Write your complete answer, including the exact file, part, or command used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

6. What is a relationship with TargetMode="External"? Give one attacker abuse (such as remote template injection) and one detection method.

Answer:

Write your complete answer, including the exact file, part, or command used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

7. Why does the PNG LSB technique fail on JPEG? Explain using the words 'lossless' and 'lossy', and name the technique steghide uses instead.

Answer:

Write your complete answer, including the exact file, part, or command used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

8. Metadata fields are trivially editable. Describe two ways to corroborate the Artist field of a photo before treating it as evidence.

Answer:

Write your complete answer, including the exact file, part, or command used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

9. zsteg prints several result lines. How do you distinguish a genuinely embedded message from coincidental bit-plane noise? Give two criteria.

Answer:

Write your complete answer, including the exact file, part, or command used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

10. Why must you hash evidence files before AND after analysis? What exactly does a mismatch at the end prove - and what does it not prove?

Answer:

Write your complete answer, including the exact file, part, or command used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

11. The steganography in this lab hid messages from humans. Which layer of an OOXML file hides content from Word's screen but not from a forensic examiner - and why is that distinction important to an investigator?

Answer:

Write your complete answer, including the exact file, part, or command used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

12. Which single finding from this lab would you present first to a manager with no technical background, and why?

Answer:

Write your complete answer, including the exact file, part, or command used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

6. Reflection and Self-Assessment

Summarize your main findings, the skills that clicked, and the obstacles you hit - with how you solved them.

Reflection:

Findings, new skills, obstacles, and how you overcame them.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

Table 6.1 — Self-assessment checklist

I can	Yes	Not yet
Explain OLE vs OOXML and unzip any Office file		
Locate text, metadata, and relationships inside the package		
Find hidden (<w:vanish/>) text and external relationships		
Triage macros with oleid and read them with olevba		
Decode an obfuscated macro payload offline in Python		
Run zsteg, interpret plane grammar, and extract the message		
Use exiftool and convert GPS to decimal degrees		
Read a spectrogram flag in Audacity		

7. Appendices

Appendix A — Tool cheat sheet

Task	Command / tool
List an Office file's parts	<code>unzip -l file.docx</code>
Extract the package	<code>unzip file.docx -d file_x/</code>
Pretty-print XML	<code>xmllint --format word/document.xml less</code>
Count hidden-text runs	<code>grep -o '<w:vanish/>' word/document.xml wc -l</code>
Triage an Office file	<code>oleid file.docm</code>
Extract VBA source	<code>olevba file.docm</code>
Full metadata dump	<code>exiftool img.jpg</code> (add <code>-a -u</code> for everything)
GPS block only	<code>exiftool -gps:all -a img.jpg</code>
Batch folder to CSV	<code>exiftool -csv -r cases/ > meta.csv</code>
PNG stego scan	<code>zsteg img.png</code> (<code>-a</code> tries all combinations)
JPEG payload	<code>steghide extract -sf cover.jpg</code> (try empty passphrase)
Headless spectrogram	<code>sox audio.wav -n spectrogram -o spec.png</code>

Appendix B — Evidence log

One row per exhibit. Keep numbering continuous across the whole session.

What good evidence looks like — model exhibit:

```
[B-04] 2026-10-07 10:12  zsteg starry_night.png
b1,rgb,lsb,xy .. text: "148:The fishermen know that
    the sea is dangerous and the storm terrible,
    but they have never found these dangers sufficient"
-> captured as evidence/B-04_zsteg_starry.png
```

- The command, the plane grammar, and the recovered text are fully visible.
- The tool and moment of capture are recorded; the filename says what it shows.
- One exhibit supports one finding.
- Bad evidence: a screenshot of a black terminal with the key line cropped out, or an answer with no exhibit number.

No.	Time	Activity / Command	Source file / Tool	Notes
B-01				
B-02				
B-03				

No.	Time	Activity / Command	Source file / Tool	Notes
B-04				
B-05				
B-06				
B-07				
B-08				
B-09				
B-10				

Appendix C — Hash verification form

Fill one row per evidence file: hash when received (start) and after analysis (end). Identical values prove faithful handling.

No	Evidence file	Algorithm	Hash (start)	Hash (end)	Match?
1					Yes / No
2					Yes / No
3					Yes / No
4					Yes / No
5					Yes / No
6					Yes / No
7					Yes / No
8					Yes / No

Appendix D — Glossary

Term	Meaning in this lab
OLE	the binary 1997-2003 Office container ('file within a file'): .doc, .ppt, .xls, .rtf
OOXML	the XML-based Office format (2007+): a ZIP of parts - .docx, .pptx, .xlsx
part	one file inside the OOXML ZIP (an XML document, an image, a binary)
[Content_Types].xml	the package's registry mapping parts to content types

Term	Meaning in this lab
docProps (core/app)	document properties: author, dates, application, template, edit time
relationship (.rels)	an XML file linking a part to its resources - internal parts or external URLs
TargetMode="External"	relationship attribute marking a target OUTSIDE the package (e.g. a remote template)
VBA macro	a Visual Basic program stored inside an Office document
AutoOpen / AutoClose	macro hooks that fire when a document is opened / closed
vbaProject.bin	the OLE binary inside m-variant Office files holding compiled VBA
oleid / olevba	oletools: triage indicators / extract full VBA source with keyword analysis
obfuscation	encoding meant to hinder reading (Chr(), Base64, hex, byte arrays) - never a match for offline decoding
steganography	hiding the EXISTENCE of a message inside an innocent carrier
LSB / bit plane	least significant bit; b1..b8 name which bit plane a scan reads
zsteg plane grammar	b1,rgb,lsb,xy = bit plane, channels, bit order, scan order
steghide	JPEG/BMP/WAV stego tool using DCT coefficient pair swapping, optionally with a passphrase
DCT	Discrete Cosine Transform - the frequency coefficients JPEG compresses
spectrogram	frequency-vs-time picture of audio; drawn text becomes readable
EXIF / metadata	data about a file: author, software, dates, camera, GPS
GPS DMS to decimal	decimal = degrees + minutes/60 + seconds/3600
carrier file	the innocent container hiding the payload
working copy	the copy you analyze; originals stay sealed and hashed
chain of custody	the documented, hash-verified handling trail of evidence

Honesty statement: by submitting this working paper I declare that all answers and evidence are my own work, that original evidence files were never modified, and that I followed the lab's ethics rules.

Student signature: _____ Date: _____

Instructor / TA signature: _____ Grade: _____